HPM6E00 系列微控制器

片内 Flash 及 MII PHY 使用指南

目录

1 简介 4

2 片内 Flash 硬件开发注意事项 5

3 片内 MII PHY 硬件开发注意事项 5

4 软件配置及生成工程 6

5 建立工程 14

6 总结 15

版本：

日期	版本号	说明
2024-08-27	1.0	初版
2024-09-29	2.0	增加 HPM6E8YIGN1 及其代码配置

1 简介

HPM6E00 系列微控制器是来自上海先楫半导体科技有限公司的高性能实时 RISC-V 微控制器，为工业自动化及边缘计算应用提供了极大的算力、高效的控制能力及丰富的多媒体功能。同时，HPM6E00 系列芯片拥有 TSN EtherCAT 等工业网络功能。

针对 Flash 和 EtherCAT 功能，HPM6E00 系列部分型号合封了 Flash 芯片及 MII PHY 芯片。本文主要提供了这类型号微控制器 Flash 及 EtherCAT 开发时注意事项。

支持合封 Flash 及 MII PHY 的 HPM6E00 系列 MCU:

序号	MCU 型号	合封 Flash	合封 MII PHY	备注
1	HPM6E8YIVM1	√	√	
2	HPM6E8YIGN1	√	√	

2 片内 Flash 硬件开发注意事项

HPM6E00 系列内置闪存型号 MCU（后文简称为本产品）在开发时，硬件设计方法与 HPM6E00 系列其他未合封型号产品相同，包括供电电路，时钟，外部存储器连接等设计注意事项，请参考相应系列产品的硬件设计指南。

本产品不需要连接外部串行 NOR Flash，因此串行总线控制器 XPI 相关的 IO 可以用于其他用途。

以 XPI0 为例，以下 IO 可以用作 GPIO 或者 XPI0 以外的其他外设复用功能：

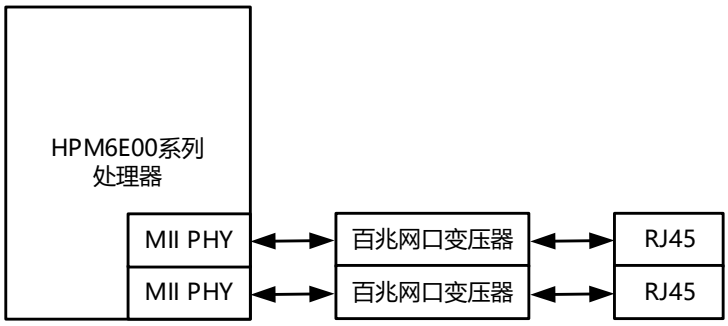
HPM6E80	信号
PB30	XPI0.CA_CS0
PB24	XPI0.CA_CS1
PB29	XPI0.CA_SCLK
PB27	XPI0.CA_D[0]
PB28	XPI0.CA_D[1]
PB26	XPI0.CA_D[2]
PB31	XPI0.CA_D[3]
PB25	XPI0.CA_DQS

用户不可以再配置串行总线控制器 XPI0 的 CA 端口，也不可以使用 XPI0 CA 端口连接外部器件。用户仍然可以配置串行总线控制器 XPI0 的 CB 端口，或者 XPI1，来连接各类外部串行存储设备。

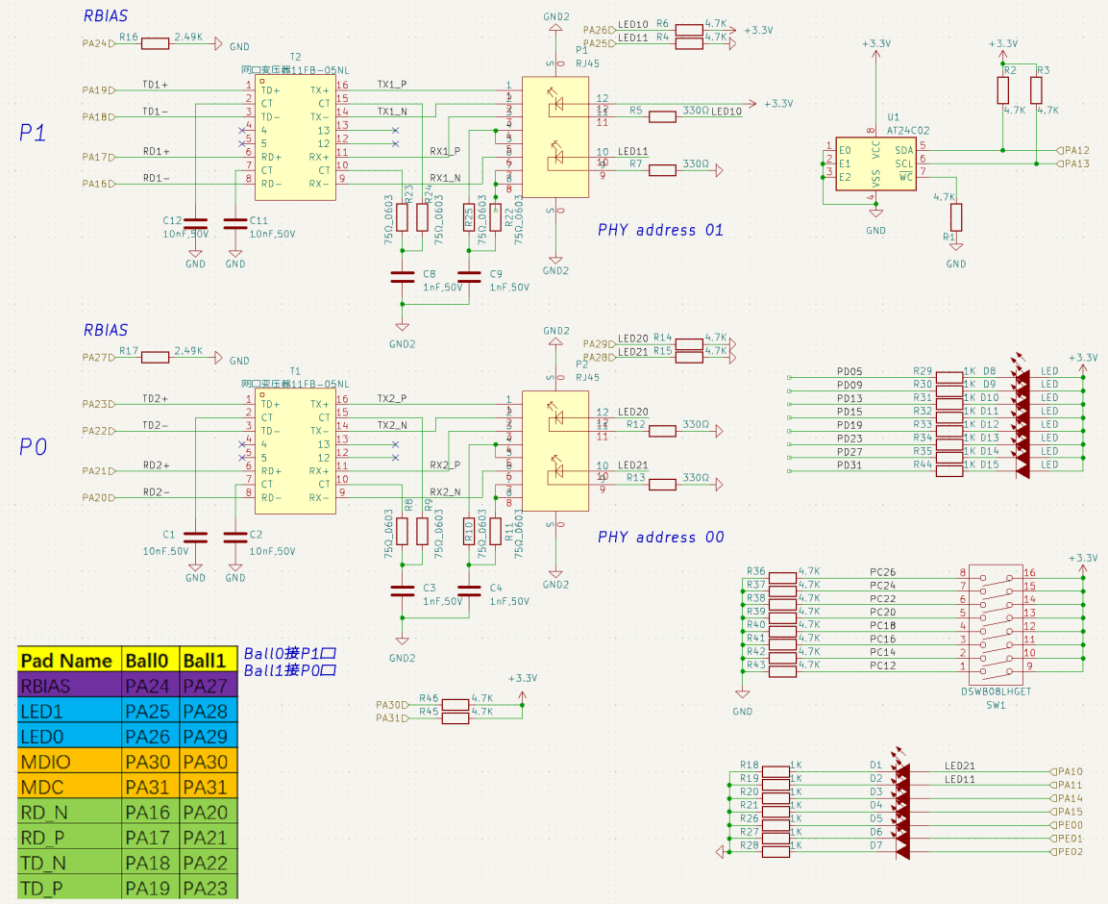
3 片内 MII PHY 硬件开发注意事项

针对 HPM6E00 系列微控制器的 EtherCAT 功能，部分型号片内合封(SIP)了两颗 MII PHY。两颗 MII PHY 在 MCU 内部，引出的 IO 直接与百兆网口变压器连接，然后通过 RJ45 连接网线。

如图：



电路参考设计：



其中，T1 和 T2 为两颗百兆网口变压器，U1 为外部 EEPROM 可以选用，如果不外挂 EEPROM，可以 Flash 模拟的 EEPROM，从而节省物料。拨码开关和 LED 为 EtherCAT 直接 IO 输入输出，可以选用。

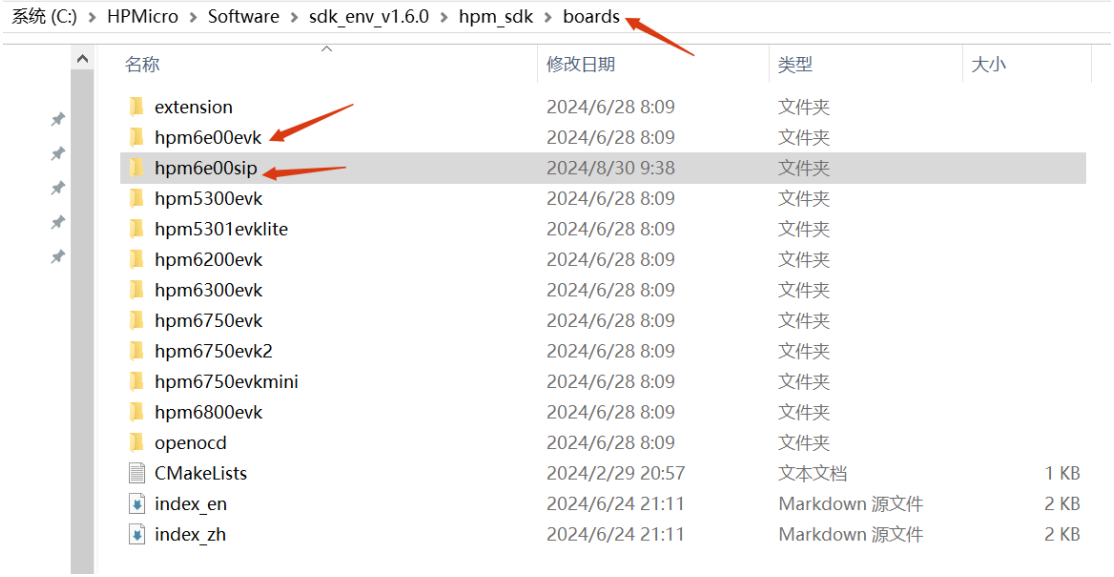
4 软件配置及生成工程

本节介绍如何在 release（v.16.0 以上）版本的 SDK 中新增 board，并使

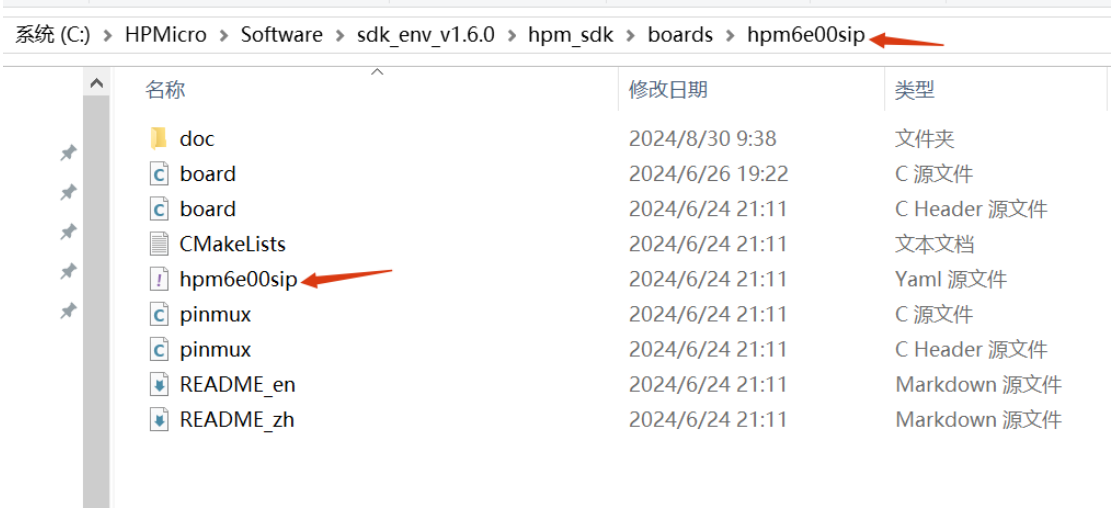
用片内 Flash 及片内 MII PHY 实现 ecat_io 例程，最新的 SDK 可以通过先楫半导体官网下载。

1）新建 board

- 复制一份 hpm6e00evk 目录并重命名为 hpm6e00sip



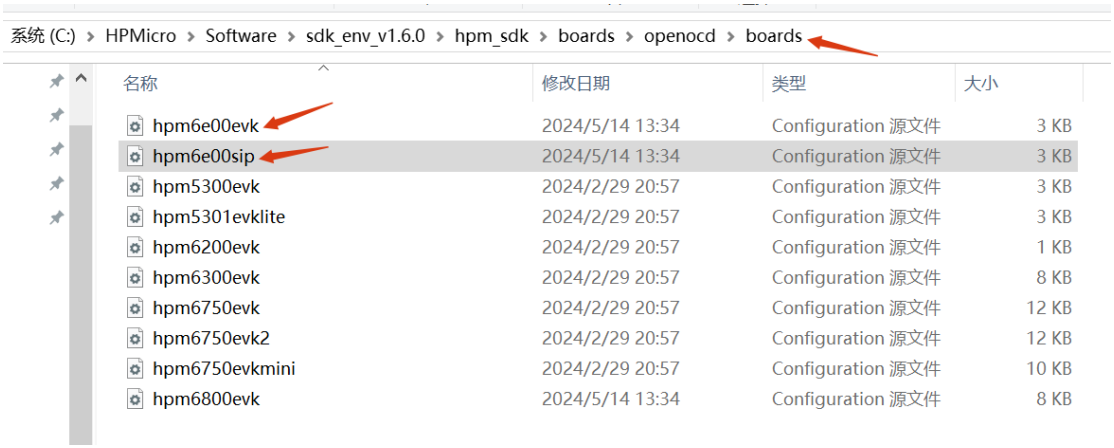
- 进入 hpm6e00sip 目录，将 hpm6e00evk.yaml 重命名为 hpm6e00sip.yaml



- 进入 hpm6e00sip.yaml，将 name 修改为 hpm6e00sip

```yaml
board:
    name: hpm6e00sip
    soc: HPM6E80
    device: HPM6E80xVMx
    openocd-soc: hpm6e80-dual-core
    openocd-probe: ft2232
    on-board-ram:
      type: sdram
      size: 32M
      width: 16bit
    on-board-flash:
      type: qspi-nor-flash
      size: 16M
    feature:
      - board_sdram
      - board_mtg
      - board_sei
```

- 进入 openocd/boards，复制一份 hpm6e00evk.cfg 并重命名为 hpm6e00sip.cfg



- board 新建完成

2）内置 Flash 软件修改

- 修改 openocd 配置文件

文件位于 **boards/openocd/boards/hpm6e00sip.cfg**， 默认是

```
flash bank xpi0 hpm_xpi 0x80000000 0x2000000 1 1 $_TARGET0 0xF3000000 0x7
```

修改为：

```
flash bank xpi0 hpm_xpi 0x80000000 0x1000000 1 1 $_TARGET0 0xF3000000 0x7 0x1000
```

- 修改 NOR Flash Configuration

文件位于 **boards/hpm6e00sip/board.c**，默认是

1. 调整 board.c 中的 NOR Flash Configuration 选项，如下：

```
#if defined(FLASH_XIP) && FLASH_XIP
__attribute__((section(".nor_cfg_option"))) const uint32_t option[4] =
{ 0xfcf90001, 0x00000007, 0x0, 0x0 };
#endif
```

修改为:

```
#if defined(FLASH_XIP) && FLASH_XIP
__attribute__((section(".nor_cfg_option"))) const uint32_t option[4] =
{ 0xfcf90002, 0x00000007, 0x1000, 0x0 };
#endif
```

修改之后，BOOTROM 启动时，会改用芯片内部的引脚去连接和引导片内的 4MB Flash。

2）内置 MII PHY 软件修改

- 修改 init_esc_pins 函数

文件位于 **boards/hpm6e00sip/pinmux.c**，将其内容替换为:

```
    HPM_IOC->PAD[IOC_PAD_PA30].FUNC_CTL = IOC_PA30_FUNC_CTL_ESC0_MDIO;
    HPM_IOC->PAD[IOC_PAD_PA31].FUNC_CTL = IOC_PA31_FUNC_CTL_ESC0_MDC;
    HPM_IOC->PAD[IOC_PAD_PA12].FUNC_CTL = IOC_PA12_FUNC_CTL_ESC0_SDA;
    HPM_IOC->PAD[IOC_PAD_PA13].FUNC_CTL = IOC_PA13_FUNC_CTL_ESC0_SCL;

    HPM_IOC->PAD[IOC_PAD_PA10].FUNC_CTL = IOC_PA10_FUNC_CTL_ESC0_CTR_1; /* NMII_LINK0 */
    HPM_IOC->PAD[IOC_PAD_PA11].FUNC_CTL = IOC_PA11_FUNC_CTL_ESC0_CTR_0; /* NMII_LINK1 */
    HPM_IOC->PAD[IOC_PAD_PA14].FUNC_CTL = IOC_PA14_FUNC_CTL_ESC0_CTR_2; /* LED_RUN */
    HPM_IOC->PAD[IOC_PAD_PA15].FUNC_CTL = IOC_PA15_FUNC_CTL_ESC0_CTR_3; /* LED_ERR */

    HPM_IOC->PAD[IOC_PAD_PW20].FUNC_CTL = IOC_PW20_FUNC_CTL_ESC0_REFCK;
    HPM_IOC->PAD[IOC_PAD_PW06].FUNC_CTL = IOC_PW06_FUNC_CTL_ESC0_P1_TXCK;
    HPM_IOC->PAD[IOC_PAD_PW11].FUNC_CTL = IOC_PW11_FUNC_CTL_ESC0_P1_TXEN;
    HPM_IOC->PAD[IOC_PAD_PW07].FUNC_CTL = IOC_PW07_FUNC_CTL_ESC0_P1_TXD_0;
    HPM_IOC->PAD[IOC_PAD_PW08].FUNC_CTL = IOC_PW08_FUNC_CTL_ESC0_P1_TXD_1;
    HPM_IOC->PAD[IOC_PAD_PW09].FUNC_CTL = IOC_PW09_FUNC_CTL_ESC0_P1_TXD_2;
    HPM_IOC->PAD[IOC_PAD_PW10].FUNC_CTL = IOC_PW10_FUNC_CTL_ESC0_P1_TXD_3;
    HPM_IOC->PAD[IOC_PAD_PW05].FUNC_CTL = IOC_PW05_FUNC_CTL_ESC0_P1_RXCK;
    HPM_IOC->PAD[IOC_PAD_PW00].FUNC_CTL = IOC_PW00_FUNC_CTL_ESC0_P1_RXDV;
    HPM_IOC->PAD[IOC_PAD_PW15].FUNC_CTL = IOC_PW15_FUNC_CTL_ESC0_P1_RXER;
    HPM_IOC->PAD[IOC_PAD_PW01].FUNC_CTL = IOC_PW01_FUNC_CTL_ESC0_P1_RXD_0;
    HPM_IOC->PAD[IOC_PAD_PW02].FUNC_CTL = IOC_PW02_FUNC_CTL_ESC0_P1_RXD_1;
    HPM_IOC->PAD[IOC_PAD_PW03].FUNC_CTL = IOC_PW03_FUNC_CTL_ESC0_P1_RXD_2;
    HPM_IOC->PAD[IOC_PAD_PW04].FUNC_CTL = IOC_PW04_FUNC_CTL_ESC0_P1_RXD_3;
```

```
HPM_IOC->PAD[IOC_PAD_PA25].FUNC_CTL = IOC_PAD_FUNC_CTL_ANALOG_MASK; /* PHY1 LED1 */
HPM_IOC->PAD[IOC_PAD_PA26].FUNC_CTL = IOC_PAD_FUNC_CTL_ANALOG_MASK; /* PHY1 LED0 */
HPM_IOC->PAD[IOC_PAD_PW12].FUNC_CTL = IOC_PW12_FUNC_CTL_GPIO_W_12; /* resetn */

HPM_IOC->PAD[IOC_PAD_PA16].FUNC_CTL = IOC_PAD_FUNC_CTL_ANALOG_MASK; /* RD_N */
HPM_IOC->PAD[IOC_PAD_PA17].FUNC_CTL = IOC_PAD_FUNC_CTL_ANALOG_MASK; /* RD_P */
HPM_IOC->PAD[IOC_PAD_PA18].FUNC_CTL = IOC_PAD_FUNC_CTL_ANALOG_MASK; /* TD_N */
HPM_IOC->PAD[IOC_PAD_PA19].FUNC_CTL = IOC_PAD_FUNC_CTL_ANALOG_MASK; /* TD_P */

HPM_IOC->PAD[IOC_PAD_PW21].FUNC_CTL = IOC_PW21_FUNC_CTL_ESC0_REFCK;
HPM_IOC->PAD[IOC_PAD_PV06].FUNC_CTL = IOC_PV06_FUNC_CTL_ESC0_P0_TXCK;
HPM_IOC->PAD[IOC_PAD_PV11].FUNC_CTL = IOC_PV11_FUNC_CTL_ESC0_P0_TXEN;
HPM_IOC->PAD[IOC_PAD_PV07].FUNC_CTL = IOC_PV07_FUNC_CTL_ESC0_P0_TXD_0;
HPM_IOC->PAD[IOC_PAD_PV08].FUNC_CTL = IOC_PV08_FUNC_CTL_ESC0_P0_TXD_1;
HPM_IOC->PAD[IOC_PAD_PV09].FUNC_CTL = IOC_PV09_FUNC_CTL_ESC0_P0_TXD_2;
HPM_IOC->PAD[IOC_PAD_PV10].FUNC_CTL = IOC_PV10_FUNC_CTL_ESC0_P0_TXD_3;
HPM_IOC->PAD[IOC_PAD_PV05].FUNC_CTL = IOC_PV05_FUNC_CTL_ESC0_P0_RXCK;
HPM_IOC->PAD[IOC_PAD_PV00].FUNC_CTL = IOC_PV00_FUNC_CTL_ESC0_P0_RXDV;
HPM_IOC->PAD[IOC_PAD_PV15].FUNC_CTL = IOC_PV15_FUNC_CTL_ESC0_P0_RXER;
HPM_IOC->PAD[IOC_PAD_PV01].FUNC_CTL = IOC_PV01_FUNC_CTL_ESC0_P0_RXD_0;
HPM_IOC->PAD[IOC_PAD_PV02].FUNC_CTL = IOC_PV02_FUNC_CTL_ESC0_P0_RXD_1;
HPM_IOC->PAD[IOC_PAD_PV03].FUNC_CTL = IOC_PV03_FUNC_CTL_ESC0_P0_RXD_2;
HPM_IOC->PAD[IOC_PAD_PV04].FUNC_CTL = IOC_PV04_FUNC_CTL_ESC0_P0_RXD_3;

HPM_IOC->PAD[IOC_PAD_PA28].FUNC_CTL = IOC_PAD_FUNC_CTL_ANALOG_MASK; /* PHY0 LED1 */
HPM_IOC->PAD[IOC_PAD_PA29].FUNC_CTL = IOC_PAD_FUNC_CTL_ANALOG_MASK; /* PHY0 LED0 */
HPM_IOC->PAD[IOC_PAD_PV12].FUNC_CTL = IOC_PV12_FUNC_CTL_GPIO_V_12; /* resetn */

HPM_IOC->PAD[IOC_PAD_PA20].FUNC_CTL = IOC_PAD_FUNC_CTL_ANALOG_MASK; /* RD_N */
HPM_IOC->PAD[IOC_PAD_PA21].FUNC_CTL = IOC_PAD_FUNC_CTL_ANALOG_MASK; /* RD_P */
HPM_IOC->PAD[IOC_PAD_PA22].FUNC_CTL = IOC_PAD_FUNC_CTL_ANALOG_MASK; /* TD_N */
HPM_IOC->PAD[IOC_PAD_PA23].FUNC_CTL = IOC_PAD_FUNC_CTL_ANALOG_MASK; /* TD_P */
```

- 使用 SIP Flash 模拟 EEPROM，需要修改 BOARD_APP_XPI_NOR_CFG_OPT_HDR

文件位于 **boards/hpm6e00sip/board.h**，默认是

```
#define BOARD_APP_XPI_NOR_CFG_OPT_HDR  (0xfcf90001U)
```

修改为：

```
#define BOARD_APP_XPI_NOR_CFG_OPT_HDR  (0xfcf90002U)
```

- 修改 EtherCAT 相关宏

文件位于 **boards/hpm6e00sip/board.h** ，默认是

```
#define BOARD_ECAT_SUPPORT_PORT1       (1)
#define BOARD_ECAT_SUPPORT_PORT2       (0)

#define BOARD_ECAT_PHY0_RESET_GPIO              HPM_GPIO0
#define BOARD_ECAT_PHY0_RESET_GPIO_PORT_INDEX   GPIO_DO_GPIOA
#define BOARD_ECAT_PHY0_RESET_PIN_INDEX         (10)

#define BOARD_ECAT_PHY1_RESET_GPIO              HPM_GPIO0
#define BOARD_ECAT_PHY1_RESET_GPIO_PORT_INDEX   GPIO_DO_GPIOA
#define BOARD_ECAT_PHY1_RESET_PIN_INDEX         (10)
#define BOARD_ECAT_PHY_RESET_LEVEL              (0)

#define BOARD_ECAT_IN1_GPIO           HPM_GPIO0
#define BOARD_ECAT_IN1_GPIO_PORT_INDEX GPIO_DO_GPIOC
#define BOARD_ECAT_IN1_GPIO_PIN_INDEX  (31U)

#define BOARD_ECAT_IN2_GPIO           HPM_GPIO0
#define BOARD_ECAT_IN2_GPIO_PORT_INDEX GPIO_DO_GPIOD
#define BOARD_ECAT_IN2_GPIO_PIN_INDEX  (9U)

#define BOARD_ECAT_OUT1_GPIO           HPM_GPIO0
#define BOARD_ECAT_OUT1_GPIO_PORT_INDEX GPIO_DO_GPIOD
#define BOARD_ECAT_OUT1_GPIO_PIN_INDEX  (8U)

#define BOARD_ECAT_OUT2_GPIO           BOARD_R_GPIO_CTRL /* reuse RGB red led */
#define BOARD_ECAT_OUT2_GPIO_PORT_INDEX BOARD_R_GPIO_INDEX
#define BOARD_ECAT_OUT2_GPIO_PIN_INDEX  BOARD_R_GPIO_PIN

#define BOARD_ECAT_OUT_ON_LEVEL        (1) /* ECAT control LED on level */

#define BOARD_ECAT_NMII_LINK0_CTRL_INDEX 3
#define BOARD_ECAT_NMII_LINK1_CTRL_INDEX 0
#define BOARD_ECAT_LED_RUN_CTRL_INDEX    1
#define BOARD_ECAT_LED_ERROR_CTRL_INDEX  6
```

修改为：

```
/* ECAT PORT0 must support */
#define BOARD_ECAT_SUPPORT_PORT1       (1)
#define BOARD_ECAT_SUPPORT_PORT2       (0)
```

```
#define BOARD_ECAT_PHY_RESET_LEVEL              (0)

#define BOARD_ECAT_PHY0_RESET_GPIO              HPM_GPIO0
#define BOARD_ECAT_PHY0_RESET_GPIO_PORT_INDEX   GPIO_DI_GPIOV
#define BOARD_ECAT_PHY0_RESET_PIN_INDEX         (12)

#define BOARD_ECAT_PHY1_RESET_GPIO              HPM_GPIO0
#define BOARD_ECAT_PHY1_RESET_GPIO_PORT_INDEX   GPIO_DI_GPIOW
#define BOARD_ECAT_PHY1_RESET_PIN_INDEX         (12)

#define BOARD_ECAT_IN1_GPIO            HPM_GPIO0
#define BOARD_ECAT_IN1_GPIO_PORT_INDEX GPIO_DO_GPIOC
#define BOARD_ECAT_IN1_GPIO_PIN_INDEX  (24U)

#define BOARD_ECAT_IN2_GPIO            HPM_GPIO0
#define BOARD_ECAT_IN2_GPIO_PORT_INDEX GPIO_DO_GPIOC
#define BOARD_ECAT_IN2_GPIO_PIN_INDEX  (26U)

#define BOARD_ECAT_OUT1_GPIO            HPM_GPIO0
#define BOARD_ECAT_OUT1_GPIO_PORT_INDEX GPIO_DO_GPIOD
#define BOARD_ECAT_OUT1_GPIO_PIN_INDEX  (5U)

#define BOARD_ECAT_OUT2_GPIO            HPM_GPIO0
#define BOARD_ECAT_OUT2_GPIO_PORT_INDEX GPIO_DO_GPIOD
#define BOARD_ECAT_OUT2_GPIO_PIN_INDEX  (9U)

#define BOARD_ECAT_OUT_ON_LEVEL         (0)

#define BOARD_ECAT_NMII_LINK0_CTRL_INDEX 1
#define BOARD_ECAT_NMII_LINK1_CTRL_INDEX 0
#define BOARD_ECAT_LED_RUN_CTRL_INDEX    2
#define BOARD_ECAT_LED_ERROR_CTRL_INDEX  3
```

上述修改以本文第 3 节《参考电路设计》为例，如果设计电路与参考电路不一致，需要根据不同硬件设计修改相应的 IO，如 IN1/2，OUT1/2，NMII_LINK0/1，LED_RUN，LED_ERROR 等。修改后，可以直接使用刚创建的 ecat_io 工程测试 EtherCAT 通信。

同样，以本文第 3 节《参考电路设计》为例，使用 HPM6E8YIGN1 型号芯片时，除了需要上述修改，还应注意：

- 修改 ecat_io 例程 hpm_ecat_hw.c 文件中 ecat_hardware_init 函数部分代码

默认：

```
esc_config_nmii_link_source(esc_ptr, true, BOARD_ECAT_SUPPORT_PORT1,
BOARD_ECAT_SUPPORT_PORT2); /* configure nmii_link source */
```

修改为：

```
esc_config_nmii_link_source(esc_ptr, false, false, false); /* configure nmii_link
source */
```

并增加代码（如果测试 P0 口通信）：

```
esc_ptr->GPR_CFG2 &= ~ESC_GPR_CFG2_NMII_LINK0_FROM_IO_MASK;
esc_ptr->GPR_CFG2 &= ~ESC_GPR_CFG2_NMII_LINK0_GPR_MASK;
```

并增加代码（如果测试 P1 口通信）：

```
esc_ptr->GPR_CFG2 &= ~ESC_GPR_CFG2_NMII_LINK1_FROM_IO_MASK;
esc_ptr->GPR_CFG2 &= ~ESC_GPR_CFG2_NMII_LINK1_GPR_MASK;
```

注意：测试口必须连接网线才能增加相应的代码，否则 EtherCAT 通讯异常。

修改后代码块（这里只测试 P0 口通信）：

```
esc_config_nmii_link_source(esc_ptr, false, false, false); /* configure nmii_link
source */
esc_ptr->GPR_CFG2 &= ~ESC_GPR_CFG2_NMII_LINK0_FROM_IO_MASK;
esc_ptr->GPR_CFG2 &= ~ESC_GPR_CFG2_NMII_LINK0_GPR_MASK;
```

HPM6E8YIGN1 型号芯片上述修改原因：

第 3 节《参考电路设计》中 phy 芯片的 link 信号连接到 PA10(P0 口 link)，PA11（P1 口 link）。HPM6E8YIGN1 型号芯片没有 PA10,PA11 引脚，只能通过上述增加代码，修改 GPR_CFG2 寄存器，以寄存器的相应 bit 位模拟 link 状态，此时不需要真实的连接信号，具体寄存器信息请参考相关文档。

本文旨在帮助读者在 SDK 基础上应用 SIP 芯片，具体设计时不建议使用模拟方式 link。

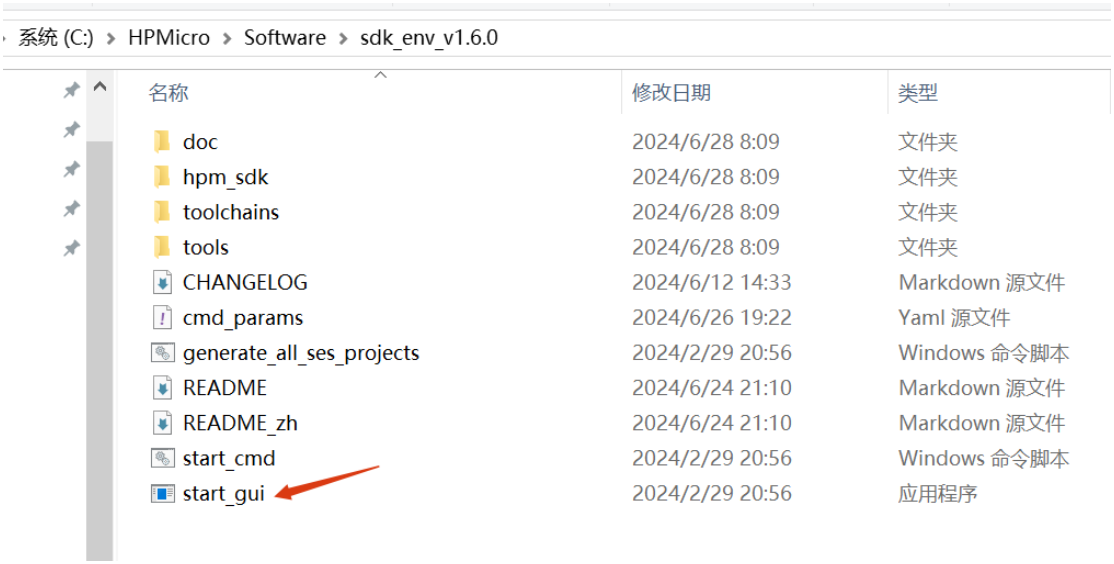
注意：生成工程时，需要保证 SSC 的代码已经存在了，否则会建立失败，具体方法参考 SDK

的 ecat_io 的 README。

5 建立工程

本节介绍使用 start_gui.exe 通过 GUI 方式生成工程，如需使用命令行生成工程请参考官网 SDK 教程。

1. 双击打开 start_gui.exe



2. 在 GUI project generator 界面中：

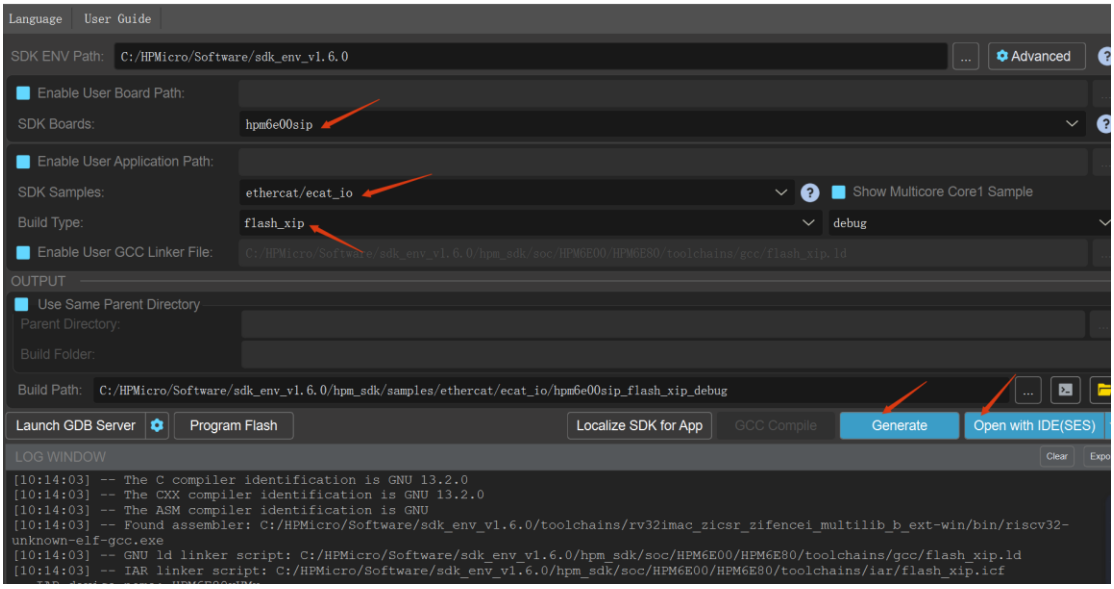
   1)、在”BOARDS”下拉列表中选”择”hpm6e00sip”

   2)、 在“SAMPLES”下拉列表中选择“ ethercat/ecat_io”

   3)、选择“flash_xip”

   4)、点击”Generate Project”按钮，生成工程

   5)、点击”Open with IDE”即可打开“ethercat/ecat_io”工程



- Flash 测试

内置 flash 和外置 flash 测试方式相同，所有使用 flash_xip 的 demo 都可以进行测试。

- EtherCAT 测试

内置 MII PHY 与外置 MII PHY 测试例程及方法相同，均可使用刚创建的 ecat_io 工程测试 EtherCAT 通信。

6 总结

HPM6E00 系列微控制器部分型号集成了 4 MB 的片内 Flash 及 MII PHY，用户不必在电路板上再安装串行 NOR Flash 及 MII PHY，可以节省开发调试成本，缩小 PCB 面积。